IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 25 August 2025, accepted 20 September 2025, date of publication 1 October 2025, date of current version 9 October 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3616323

RESEARCH ARTICLE

Point Cloud Upsampling for Accurate Surface Reconstruction via Attention-Guided Generation

JONG-SU WON[1] AND JONG-KI HAN[2]
[1]Department of Information and Communications Engineering, Sejong University, Gwangjin-gu, Seoul 05006, South Korea
[2]Department of AI Convergence Electronics Engineering, Sejong University, Gwangjin-gu, Seoul 05006, South Korea

Corresponding author: Jong-Ki Han (hjk@sejong.edu)

This work was supported by the Faculty Research Fund of Sejong University, in 2025.

ABSTRACT Reconstructing accurate surfaces from sparse point clouds remains a fundamental challenge in 3D vision. In this paper, we propose a novel upsampling framework, called Surf-PU, that focuses on surface-aware point generation. Our method comprises two key components. First, we introduce an adaptive query generation module that analyzes local attention information to determine where new points should be placed. These query points guide the generation toward regions of geometric significance. Second, we incorporate the Hyper Chamfer Distance (HCD) loss, which accounts for the distribution of point-wise distances to better capture complex surface structures. Unlike existing methods, our approach effectively combines attention-based guidance and loss-driven precision, leading to more accurate surface reconstruction. Extensive experiments on PU-GAN and PU1K datasets demonstrate that our method consistently outperforms state-of-the-art techniques, especially in terms of the P2F metric. Moreover, the proposed framework maintains robust performance across arbitrary upsampling ratios and under random noise, confirming its reliability and generalizability.

INDEX TERMS Attention mechanisms, chamfer distance, point cloud upsampling, surface reconstruction.

I. INTRODUCTION

Point clouds are collections of 3D coordinates obtained from sensors such as LiDAR and RGB-D cameras. In practice, these point clouds are often sparse, noisy, non-uniform, and prone to occlusion. To enable the use of point clouds in downstream tasks such as segmentation [1], surface reconstruction [2], autonomous driving [3], and robotic analysis [4], it is crucial to obtain high resolution point clouds from raw low-resolution data. Point cloud upsampling is not simply about increasing the number of points. It also plays a critical role in learning-based data processing by removing noise and reconstructing occluded object parts.

Point cloud upsampling methods can be broadly classified into two categories: optimization-based and deep learning-based approaches. Traditional optimization-based methods [5], [6], [7], [8], [9], [10] estimate high-resolution point clouds from sparse inputs by relying on geometric priors such as surface normals, curvature, and local point density. While these methods perform well in flat regions, they have limitations in reconstructing complex structures or sharp features. To overcome these challenges, recent approaches have shifted toward deep learning-based methods [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28] that learn meaningful features directly from data. However, since point clouds are irregularly distributed in 3D space and lack an inherent ordering, directly applying conventional 2D grid-based deep learning models is not straightforward. To address this, prior point cloud upsampling studies have introduced techniques such as PointNet++ [29], graph convolution [30], attention mechanisms [31], kernel point convolution (KPConv) [32], and 4D point convolution [33] to extract meaningful features and design effective upsampling modules.

Some studies have attempted to capture local geometric relationships by identifying neighboring points around a center point. However, these approaches often overly rely on neighbors, causing the features of center points and their

The associate editor coordinating the review of this manuscript and approving it for publication was Lorenzo Mucchi.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

neighbors to become indistinguishable, which can result in overlapping generated points. To mitigate this problem, repulsion loss [11], uniform loss [13], and techniques like FPS (farthest point sampling) [34] have been employed. Nevertheless, existing approaches are still susceptible to noise and struggle to accurately reconstruct object surfaces, which remains the most critical goal in point cloud upsampling.

RepKPU [27] addressed this limitation by extracting local geometric features for each input point using deformable kernel point convolution, storing the locations and features of the convolution kernels as RepKPoints to represent the geometric structure of neighboring points. Unlike previous methods, RepKPU assigns independent kernel points and features to each input point, enabling better distinction between the center point and its neighbors.

In RepKPU, four predefined positions are used as kernel point queries to generate new points. The features of these kernel point queries are extracted through cross-attention operations, where the kernel point queries serve as queries and the RepKPoints serve as keys and values. This process produces displacement features, which are then mapped to displacement vectors through multilayer perceptrons (MLPs). The input points are replicated four times, and each replica is added to its corresponding displacement vector to perform upsampling. However, because the kernel point queries are fixed in positions relative to the input points, they struggle to produce adaptive displacement vectors across varying data distributions. This limitation becomes particularly evident in edge regions or complex structures, where the fixed kernel point queries fail to adequately capture local geometric features. As a result, the upsampled point cloud may exhibit rough surfaces and lower point-to-surface (P2F) [35] accuracy.

To address these limitations and reconstruct smoother and more accurate surfaces in high-resolution point clouds, this paper proposes two methods. First, instead of simply shifting the fixed kernel point queries, which can be unstable due to the lack of ground truth for optimal positions, we propose an alternative approach. We emphasize the features of neighboring points that better reflect geometric relations when projecting them onto the kernel point query features. By adjusting the weighting functions used in this projection, we achieve an effect similar to adaptively shifting kernel point queries.

To find neighbors that accurately capture geometric relations, we leverage vector attention operator proposed in Point Transformer [31]. The attention scores obtained from vector attention represent the geometric relations between input and neighboring points. By incorporating these attention scores into the distance-based weighting function for projecting neighbor features onto kernel point query features, we enhance the contribution of geometrically relevant features. As a result, the model generates adaptive displacement vectors that more accurately reflect the geometry of the local region.

The second proposed method introduces the Hyperbolic Chamfer Distance loss [36]. Chamfer Distance loss [37], widely used for training deep learning models in point cloud upsampling, calculates the mean distance between each point in the output and its nearest point in the ground truth point cloud. However, Chamfer Distance loss can be misled by incorrect nearest matches and is vulnerable to noise. Recent studies [38], [39], [40] have highlighted the effectiveness of hyperbolic spaces in representing non-Euclidean structures such as tree-like data. Hyperbolic Chamfer Distance loss amplifies the contributions of closer point pairs while suppressing those of distant pairs. This property enhances robustness to noise and improves accuracy in edge and surface reconstruction. In this paper, we demonstrate that using Hyperbolic Chamfer Distance loss enables more accurate surface predictions.

The contributions of this paper are as follows. First, we analyze and visualize the limitations of previous upsampling methods. Second, we propose an adaptive KP-Queries Generation Module to overcome the shortcomings in surface reconstruction capabilities. Third, by reusing attention scores, our Adaptive KP-Queries Generation Module achieves meaningful performance gains while incurring no extra GPU memory usage. Fourth, to the best of our knowledge, this is the first application of Hyperbolic Chamfer Distance loss to point cloud upsampling, which leads to more accurate surface reconstruction, particularly around edges and complex structures.

The remainder of this paper is organized as follows. Section II reviews previous point cloud upsampling methods. Section III details the proposed deep learning model and loss function. Section IV presents and analyzes quantitative and qualitative experimental results. Section V concludes the paper.

II. RELATED WORK

A. OPTIMIZATION-BASED METHODS

Optimization-based methods laid the groundwork for point cloud upsampling before the rise of deep learning. These approaches primarily rely on explicit geometric priors to reconstruct high-resolution point clouds from sparse inputs. A Voronoi-based method [5] increases point density while preserving local surface smoothness by inserting new points at the vertices of Voronoi diagrams computed in the local tangent space. The Locally Optimal Projection (LOP) operator [6] projects input points onto an estimated surface using the L1 median. This approach offers robustness to noise and outliers, enabling consistent point resampling and surface reconstruction. To address non-uniformity and improve edge preservation, a weighted variant of LOP was proposed in [7], incorporating local point densities into the projection process. The Edge-Aware Resampling (EAR) method [8] gradually resampled points from smooth areas toward edge singularities, effectively preserving sharp features in the upsampled point set. Other approaches aimed at improving

computational efficiency and robustness. For instance, [9] developed fast approximations of LOP to reduce computational cost, while [10] proposed point set consolidation methods that enrich surface points with deep representations to fill large missing regions.

Despite their advantages, optimization-based methods have inherent limitations. They often assume that underlying surfaces are smooth and continuous, making them less effective for highly noisy, sparse, or complex structures. Additionally, these methods depend on accurate normal estimation, which can be sensitive to noise and scanning artifacts. Their iterative nature also leads to high computational costs, limiting scalability for large-scale real-world point clouds.

B. DEEP LEARNING-BASED METHODS

With the rapid advancement of deep learning, numerous data-driven approaches have been proposed for point cloud upsampling. PU-Net [11] utilized PointNet++ [29] as its backbone to extract multi-scale features and expanded the point set through multi-branch MLPs, although it often produced clustered and redundant points. MPU [12] employed a feature expansion strategy using an EdgeConv [41] based feature extractor and multi-step upsampling to better preserve details. PU-GAN [13] adopted a generative adversarial network (GAN) [42] framework to improve the uniformity of the upsampled points and introduced the uniform loss to enforce even point spacing. PU-Geo [14] learned a mapping function to transform 2D samples into 3D space, enabling geometry-aware upsampling. Dis-PU [15] proposed a coarse-to-fine architecture combining a dense generator and a spatial refiner to improve output quality. PU-GCN [16] leveraged Inception DenseGCN [30] and a NodeShuffle module to effectively capture local geometric features. PU-Transformer [17] applied transformer-based attention modules to learn fine-grained features for high-fidelity upsampling. MAFU [18] performed midpoint interpolation followed by refinement to support flexible upsampling ratios. PU-Refiner [19] introduced a GAN-based architecture that includes a transformer-based geometry refiner within a coarse-to-fine framework to enhance geometric accuracy. SeedFormer [20] also employed a coarse-to-fine structure, utilizing transformer-based seed point refinement to progressively improve results. PUFA-GAN [21] combined feature attention with a GAN framework and effectively suppressed high-frequency noise, resulting in sharper and cleaner upsampling outputs. Neural Points [22] represented point clouds as continuous neural fields, enabling arbitrary-scale upsampling. PU-Flow [23] formulated upsampling as a normalizing flow problem, learning invertible mappings between Euclidean and latent spaces. PU-Dense [24] employed a U-Net architecture with sparse convolutional layers for point feature extraction, resulting in a sparse tensor-based representation that efficiently supports large-scale point cloud upsampling. PC2-PU [25] introduced patch correlation and point correlation modules to maintain global spatial consistency. Grad-PU [26] generated a coarse point cloud via midpoint interpolation, which was then fed into P2PNet to predict point-to-point distances with the ground truth, formulating upsampling as a gradient-based refinement problem. RepKPU [27] extracted kernel point representations using deformable kernel point convolution and performed upsampling through a kernel-to-displacement generation framework. PU-Mask [28] treated upsampling as a local filling problem, employing a masking strategy to enhance robustness in complex real-world scenarios.

While these deep learning-based methods have achieved remarkable performance improvements, most rely on repulsion loss and uniform loss functions to prevent overlapping points in the upsampled outputs. Although these losses encourage uniform point distributions, they can inadvertently hinder the ultimate goal of point cloud upsampling, which is to accurately reconstruct the surface geometry of the object.

To address this limitation, we adopt the Hyper Chamfer Distance loss, which better approximates curved surfaces and improves surface fidelity. Furthermore, we propose an Adaptive KP-Queries Generation Module that emphasizes points with rich local geometric features to generate data-adaptive queries, thereby enhancing surface reconstruction accuracy.

III. PROPOSED ALGORITHM

The proposed model consists of the Point Transformer Encoder, the Representation Kernel Points Extraction Module, the Adaptive Kernel Point Queries Generation Module, and the Multi-head Cross-Attention Module. The overall architecture is illustrated in Fig. 1 The following sections detail the roles and computational processes of each module.

A. POINT TRANSFORMER ENCODER

The Point Transformer Encoder, originally introduced in Point Transformer [31], takes an input point cloud $P \in \mathbb{R}^{N\times 3}$ and extracts per-point features $F_e \in \mathbb{R}^{N\times C}$ using vector attention operations. The computation is defined as:

$$f_e = \sum\nolimits_{p_i \in K(p)} o_i \odot (\alpha\,(p_i) + \delta) \tag{1}$$

where $f_e \in F_e$, $p \in P$, and $K(p) \in \mathbb{R}^{N_{knn}\times 3}$ is its k-nearest neighbors, with N_{knn}= 16 in this study. $\odot$ denotes the Hadamard product, α denotes a linear projection function, and δ denotes the positional encoding. The attention score $O \in \mathbb{R}^{N\times N_{knn}\times C}$ is computed as:

$$o_i = \rho(\gamma\,(\varphi\,(p) - \psi\,(p_i) + \delta)), p_i \in K(p) \tag{2}$$

where $o_i \in O$, ρ denotes the softmax function, γ denotes a mapping function, and φ, ψ denote linear projections. The positional encoding δ is based on the relative coordinates between the center point and its neighbors:

$$\delta = \theta(p - p_i) \tag{3}$$

where θ is the encoding function.

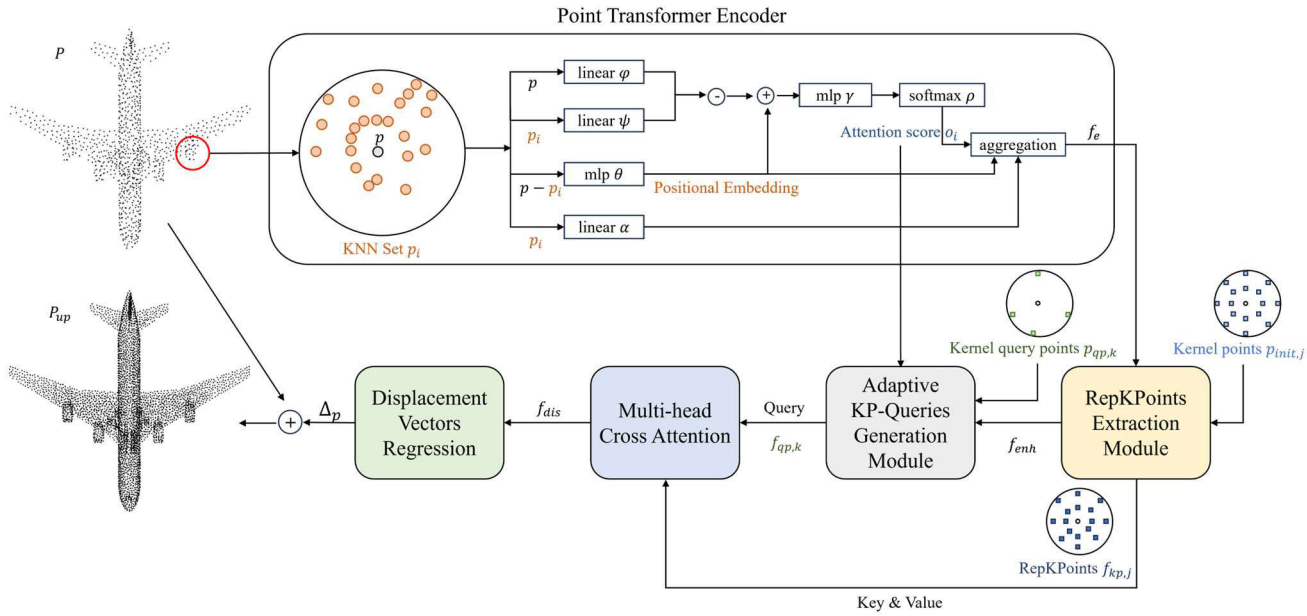

FIGURE 1. Overall architecture of the proposed point cloud upsampling framework. The network takes an input point cloud and its features, and extracts local attention scores using a vector attention mechanism. These scores guide the Adaptive KP-Queries Generation Module to produce kernel point queries in geometrically important regions. The kernel point query features are then combined with learnable weights and passed to the RepKPoints Extraction Module, which encodes kernel point representations. The Multi-head Cross Attention and Displacement Vectors Regression Modules generate the upsampled point cloud.

This module encodes both the relative distance and feature differences between a center point and its neighbors,producing features F_e that effectively capture the geometric structure of the input point cloud. Moreover, the attention scores O derived from vector attention are used in the subsequent Adaptive KP-Queries Generation Module to generate kernel point query features that reflect the geometric significance of neighboring points.

B. REPKPOINTS EXTRACTION MODULE

The RepKPoints Extraction Module, originally proposed in RepKPU [27], takes the input point cloud P and its features F_e to generate local features $F_{enh} \in \mathbb{R}^{N \times C_{kp}}$ and RepKPoints $F_{kp} \in \mathbb{R}^{N \times N_{kp} \times C_{kp}}$ using deformable kernel point convolution operations. RepKPoints transform the local geometric structures of the input points into a kernel point–based representation, playing a crucial role in subsequent query generation and displacement prediction. The deformable kernel point convolution consists of two operations. First, point offsets are predicted using a predefined rigid kernel point convolution. Second, the offsets are added to the kernel points, and features are extracted using the deformable kernel point convolution. The initial kernel points $P_{init} \in \mathbb{R}^{N_{kp} \times 3}$ are uniformly placed on the surface of a sphere with radius r_{kp} centered at each input point. Unlike 2D image convolutions, neighboring points are not necessarily located at the exact positions of P_{init}. Therefore, RepKPU [27] employs a projection function that maps the features of neighboring points onto the kernel point features. Using a ball query with a radius r_s, we select N_{ball} neighboring points for each query point. The projection function h between initial kernel point and its neighboring point is defined as:

$$h(p_i, p_{init,j}) = (0, 1 - \frac{\|(p - p_i) - p_{init,j}\|}{\sigma}) \tag{4}$$

where $p_{init,j} \in P_{init}$, $p \in P$, $p_i \in Q(p) \in \mathbb{R}^{N_{ball} \times 3}$ is its ball query neighbors. σ is the receptive field.

The projected kernel point features $F_p \in \mathbb{R}^{N \times N_{kp} \times C_{kp}}$ are then computed as:

$$f_{p,j} = \sum\nolimits_{p_i \in Q(p)} h\left(p_i, p_{init,j}\right) f_{e,i} \tag{5}$$

where $f_{p,j} \in F_p$ and $f_{e,i} \in F_e$ is the neighbor feature. These features are then multiplied by learnable weights $W_o \in \mathbb{R}^{N_{kp} \times C_{kp} \times C}$ to produce the rigid kernel point convolution output features $F_{out} \in \mathbb{R}^{N \times C_{kp}}$:

$$f_{out} = \sum\nolimits_{j=0}^{N_{kp}-1} f_{p,j} w_{o,j} \tag{6}$$

where $f_{out} \in F_{out}, w_{o,j} \in W_o$. Then, the predicted kernel point offsets Δ_{kp} are obtained by passing F_{out} through an MLP:

$$\Delta_{kp} = MLP(f_{out}) \tag{7}$$

The deformable kernel points $P_{kp} \in \mathbb{R}^{N_{kp} \times 3}$ are obtained by adding the kernel point offset $\Delta_{kp} \in \mathbb{R}^{N_{kp} \times 3}$ to P_{init}. The deformed kernel point features ΔF_p are defined as:

$$\Delta f_{p,j} = \sum\nolimits_{p_i \in Q(p)} h\left(p_i, p_{kp,j}\right) f_{e,i} \tag{8}$$

where $\Delta f_{p,j} \in \Delta F_p, p_{kp,j} \in P_{kp}$.

The local features $f_{enh} \in F_{en}$ are then calculated as:

$$f_{enh} = \sum_{j=0}^{N_{kp}-1} \Delta f_{p,j} w_{enh,j} \tag{9}$$

where $w_{enh,j} \in W_{enh} \in \mathbb{R}^{N_{kp} \times C_{kp} \times C}$ are learnable weights. Then, the RepKPoints $f_{kp,j} \in F_{kp}$ are obtained by concatenating ΔF_p and F_{enh}, followed by an MLP:

$$f_{kp,j} = MLP\left(cat\left(\Delta f_{p,j}, f_{enh}\right)\right) \tag{10}$$

The hyperparameter settings ($N_{kp} = 15$, $C = 64$, $C_{kp} = 128$, $\sigma = 0.2$, $r_s = 4\sigma$, $r_{kp} = 1.5\sigma$, $N_{ball} = 30$) are validated values from KPConv [32] and RepKPU [27], chosen to balance local neighborhood coverage and computational efficiency.

The RepKPoints Extraction Module produces independent kernel point features that reflect diverse local geometric structures for each input point, enabling precise surface reconstruction in the subsequent Adaptive KP-Queries Generation Module and Multi-head Cross-Attention.

C. ADAPTIVE KP-QUERIES GENERATION MODULE

The Adaptive KP-Queries Generation Module takes the input features F_{enh}, the fixed kernel point queries $P_{qp} \in \mathbb{R}^{r \times 3}$, and the attention scores O to generate r adaptive KP-Query features $F_{qp} \in \mathbb{R}^{N \times r \times C_{kp}}$ using kernel point convolution-style weighting operations. r is the upsampling ratio, which is fixed to 4 in this study. The entire computation flow of this module is illustrated in Fig. 2.

In RepKPU [27], the projection function h between the kernel point query and its neighbor point is solely used to project neighbor point features onto the kernel point query features. However, when the kernel point query locations are fixed, these distance-based weights can give undue influence to neighbors with low attention scores, often resulting in displacement vectors that poorly reflect local geometric structures and producing rough surfaces in the upsampled point cloud. While it is possible to address the limitation of fixed kernel point queries by learning to adaptively shift their locations to fit local geometric structures, such an approach is challenging due to the absence of ground truth for optimal positions and the instability in training. Therefore, we take an alternative approach. Instead of moving the kernel point queries themselves, we design an adaptive projection function by incorporating attention scores, which encode the local geometric importance, into the distance-based weight function. This approach has a similar effect to adaptively shifting the kernel point queries while enabling more stable training, as it avoids directly optimizing their positions. The proposed adaptive projection function h_{adap} is defined as:

$$h_{adap,i,k}(p_i, p_{qp,k}) = max(0, 1 - \frac{\left\|(p - p_i) - p_{qp,k}\right\|}{\sigma}) + \lambda o_i^{masked} \tag{11}$$

where $p \in P$, $p_i \in Q(p)$, $p_{qp,k} \in P_{qp}$, and λ is a hyperparameter that controls the influence of the attention scores.

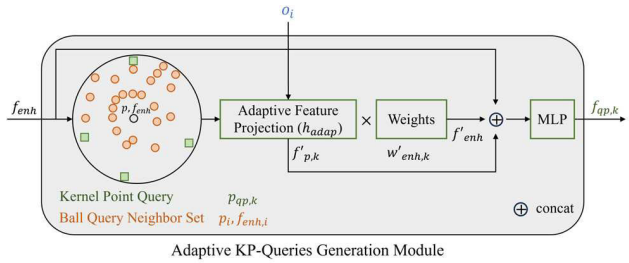

FIGURE 2. Architecture of the adaptive KP-Queries generation module.

The masked attention score $o_i^{masked} \in O^{masked}$ is defined as follows:

$$o_i^{masked} = \begin{cases} o_i, & if\ o_i > \bar{O}, \\ 0, & otherwise. \end{cases} \tag{12}$$

where $\bar{O} = \frac{1}{N_{knn}}$ is the expected attention score for each neighbor, derived from the fact that the softmax operation enforces $\sum_i o_i = 1$. This threshold reduces the influence of low-confidence neighbors and emphasizes the contribution of high-confidence ones during the adaptive query features generation. The distance-based weights and attention scores are then combined to further strengthen the impact of these emphasized neighbors.

The computational process of the Adaptive KP-Queries Generation Module is similar to that of the RepKPoints Extraction Module. First, the proposed h_{adap} is used to project the neighbor point features onto the adaptive kernel point query features $F'_p \in \mathbb{R}^{N \times r \times C_{kp}}$:

$$f'_{p,k} = \sum_{p_i \in Q(p)} h_{adap,i,k}\left(p_i, p_{qp,k}\right) f_{enh,i} \tag{13}$$

where $f'_{p,k} \in F'_p$ and $f_{en,i}$ is the neighbor feature. Then, by multiplying with learnable weights $W'_{enh} \in \mathbb{R}^{N_{kp} \times C_{kp} \times C_{kp}}$, the local features $F'_{enh} \in \mathbb{R}^{N \times C_{kp}}$ is computed as:

$$f'_{en} = \sum_{k=0}^{N_{kp}-1} f'_{p,k} w'_{enh,k} \tag{14}$$

where $w'_{enh,k} \in W'_{enh}$, $f'_{enh} \in F'_{enh}$. The Adaptive KP-Query features $f_{qp,k} \in F_{qp}$ is obtained by concatenating F_{enh},F'_{enh},F'_p, followed by an MLP:

$$f_{qp,k} = MLP\left(cat\left(f'_{p,k}, f_{enh}, f'_{en}\right)\right) \tag{15}$$

This module allows the generation of data-adaptive KP-Query features F_{qp} that reflect local geometric information, rather than relying solely on fixed-location queries. As a result, more precise displacement vector prediction is possible in the subsequent cross-attention stage.

D. MULTI-HEAD CROSS-ATTENTION

The Multi-head Cross-Attention Module takes the RepKPoints features F_{kp} and the Adaptive KP-Queries features F_{qp} as inputs and outputs the displacement features $F_{dis} \in \mathbb{R}^{(N \times r) \times C_{kp}}$. Following the general structure of multi-head cross-attention proposed in [43], F_{kp} serves as both the Key

and Value, while F_{qp} is used as the Query for the cross-attention operation. In this study, the feature dimensions of Query, Key, and Value are all set to 128, ensuring that attention weight computation and feature aggregation are performed in a consistent space. Through the cross-attention operation, the Adaptive KP-Query features reference the geometric information encoded in the RepKPoints to generate displacement features $f_{dis} \in F_{dis}$ for each point queries. The computation process is defined as follows:

$$f_{dis} = CrossAttention(q, k, v) \tag{16}$$

Here, $CrossAttention(q, k, v)$ computes multi-head attention weights using the projected Query q, Key k, and Value v, and generates output features through a weighted sum.

The generated displacement features f_{dis} are passed through an MLP to approximate the displacement vectors Δ_p. For each input point, r per-point displacement vectors Δ_p are generated, and the upsampled point cloud $P_{up} \in \mathbb{R}^{(N\times r)\times 3}$ is obtained by replicating the input point cloud P r times and adding each displacement vector. This can be expressed as:

$$P_{up} = repeat(P, r) + \Delta_p \tag{17}$$

Here, $repeat(P, r)$ denotes the operation of replicating the input point cloud Pr times.

This module enables the Adaptive KP-Queries to generate displacement vectors that reflect the local geometric representation encoded in the RepKPoints. As a result, the upsampled point cloud better preserves the geometric structure of the input.

E. TRAINING LOSS

In this paper, we employ a composite loss function consisting of three components. The first component is the reconstruction loss, defined by the Hyperbolic Chamfer Distance [36]. Given the ground truth point cloud P_{gt} and the predicted point cloud P_{up}, the loss is formulated as:

$$L_{recon} = \sum_{x\in P_{gt}} \min_{y\in P_{up}} arcosh\left(1 + \epsilon\|x-y\|_2^2\right) + \sum_{y\in P_{up}} \min_{x\in P_{gt}} arcosh\left(1 + \epsilon\|y-x\|_2^2\right) \tag{18}$$

where the scalar ϵ is defined as:

$$\epsilon = \frac{2}{(1-\|x\|^2)(1-\|y\|^2)} \tag{19}$$

For reference, the Chamfer Distance loss is defined as:

$$L_{CD} = \sum_{x\in P_{gt}} \min_{y\in P_{up}} \|x-y\|_2^2 + \sum_{y\in P_{up}} \min_{y\in P_{gt}} \|y-x\|_2^2 \tag{20}$$

Compared to the Chamfer Distance loss, the Hyperbolic Chamfer Distance loss more heavily penalizes close point pairs and reduces the influence of distant pairs. This leads to improved robustness on surface edges and curved regions, while reducing sensitivity to noise.

Additionally, we adopt the fitting loss L_{fit} and repulsion loss L_{rep} as regularization terms, both originally proposed in KPConv [32]. The fitting loss addresses the gradient vanishing problem caused by the sparsity of point clouds. It encourages each deformable kernel point to move closer to its nearest neighboring point in the input. However, relying solely on fitting loss may cause all kernel points to collapse toward the same neighbor. To mitigate this, the repulsion loss is employed to ensure that the deformable kernel points maintain sufficient separation and are uniformly distributed. The total training loss is defined as:

$$L = L_{recon} + L_{fit} + L_{rep} \tag{21}$$

IV. EXPERIMENTAL RESULTS

A. DATASETS

In this study, we employ two widely adopted benchmark datasets for point cloud upsampling: PU-GAN [13] and PU1K [16]. Both datasets are constructed from high-quality 3D mesh models and follow a standardized protocol based on Poisson disk sampling [44] to generate point clouds for training and evaluation.

The PU-GAN dataset consists of 120 CAD mesh models for training and 27 for testing. A total of 24,000 training patches are sampled from these meshes, where each patch contains 256 sparse input points and 1,024 dense ground truth points, corresponding to a 4× upsampling ratio. During testing, each input point cloud contains 2,048 points, and the corresponding ground truth contains 8,192 points. In addition to the standard 4× upsampling, we also conduct evaluations under 16× upsampling, as well as under arbitrary upsampling ratios including 5×, 6×, 7×, 11×, 13×, and 15× to verify the flexibility of our model. Furthermore, to evaluate robustness, we introduce random noise into the input point clouds and analyze performance degradation under noisy conditions.

The PU1K dataset is a larger and more diverse benchmark built from a collection of 1,020 mesh models derived from both PU-GAN and ShapeNetCore [45] datasets, with an additional 127 models for testing. It spans over 50 object categories, thereby ensuring a broad spectrum of geometric complexity. A total of 51,000 patches are generated for training, each consisting of 256 input points and 1,024 ground truth points. To enhance generalization, we apply data augmentation techniques such as random scaling, rotation, and point perturbation during training. For evaluation, each 2,048-point input cloud is split into overlapping patches of 256 points. Each patch is independently upsampled, and then the outputs are merged into a dense point cloud of 8,192 points using FPS (Farthest Point Sampling) to ensure spatial uniformity and minimal redundancy. This procedure strictly follows the protocol established in prior studies [26], [27]. Additionally, we qualitatively assess our method using unseen shapes from the large-scale ShapeNetCore dataset, demonstrating that the proposed method generalizes well to complex and diverse object categories.

TABLE 1. Quantitative results for 4× and 16× upsampling on the PU-GAN dataset.

Methods	4 ×				16 ×			Params
	CD↓ $\times 10^3$	HD↓ $\times 10^2$	P2F↓ $\times 10^3$	Speed↓ ms/pcd	CD↓ $\times 10^3$	HD↓ $\times 10^2$	P2F↓ $\times 10^3$	
PU-Net[11]	0.482	4.671	8.879	130	0.450	7.387	12.146	0.54 M
PU-GAN[13]	0.260	4.707	1.990	171	0.193	6.213	2.589	0.81 M
Dis-PU[15]	0.264	4.410	2.019	251	0.133	5.711	2.230	1.04 M
PU-GCN[16]	0.257	**1.957**	2.878	141	0.110	**1.958**	3.286	0.07 M
PUFA-GAN[21]	0.247	4.847	1.956	337	0.118	5.159	2.130	4.64 M
PU-Mask[28]	0.266	3.349	1.939	295	0.168	4.581	2.176	1.02 M
Grad-PU[26]	0.264	**2.623**	1.982	128	0.129	**2.631**	2.242	0.06 M
RepKPU[27]	**0.248**	2.880	**1.906**	58	**0.107**	3.345	**2.068**	1.01 M
Ours	**0.243**	2.704	**1.834**	**62**	**0.109**	3.275	**1.873**	1.01 M

Bold red denotes the best value, and blue denotes the second-best value.

B. IMPLEMENTATION DETAILS

All experiments in this study were conducted on an NVIDIA RTX 4090 GPU. Training was performed for 150 epochs with a batch size of 32, using the Adam optimizer [46]. The initial learning rate was set to $1e^{-3}$, and it was gradually decayed to $1e^{-4}$ throughout the training process. To ensure a fair comparison, we adopted the same data augmentation strategies as in prior works [26], [27] including random scaling, rotation, and jittering.

For the Adaptive KP-Queries Generation Module, the weighting coefficient λ was empirically set to 3 for the PU-GAN dataset and 4 for the PU1K dataset, reflecting the varying complexity and distribution of the datasets. The total training time was approximately 5 hours for PU-GAN and 14.5 hours for PU1K on an NVIDIA RTX 4090 GPU.

C. EVALUATION METRIC

To quantitatively evaluate upsampling performance, we adopted three standard metrics: Chamfer Distance (CD) [37], Hausdorff Distance (HD) [48], and Point-to-Surface Distance (P2F) [35]. The Chamfer Distance measures the average closest-point distance between the predicted and ground truth point clouds, reflecting the overall shape similarity. The Hausdorff Distance captures the largest distance between the two sets, making it sensitive to outliers and local errors. The Point-to-Surface Distance evaluates how well the upsampled points approximate the ground truth mesh surface. While CD and P2F provide average-case measures that capture the overall distribution and surface fidelity, HD is dominated by the single worst-case point pair and therefore may exaggerate local deviations without fully reflecting visual quality. For all three metrics, lower values indicate better upsampling quality.

TABLE 2. Quantitative results for 4× upsampling on the PU1K dataset.

Methods	CD↓ $\times 10^3$	HD↓ $\times 10^2$	P2F↓ $\times 10^3$
PU-Net[11]	0.665	0.693	5.747
MPU[12]	0.440	0.340	1.237
PU-GAN[13]	0.424	0.540	1.580
Dis-PU[15]	0.394	0.362	1.336
PU-GCN[16]	0.385	0.374	1.385
MAFU[18]	0.362	0.450	1.139
PU-Flow[23]	0.356	0.321	1.315
Grad-PU[26]	0.404	0.373	1.474
RepKPU[27]	**0.327**	**0.268**	**0.938**
Ours	**0.316**	**0.265**	**0.864**

Bold red denotes the best value, and blue denotes the second-best value.

D. QUANTITATIVE EVALUATION

We first evaluated our method on the PU-GAN dataset using 4× and 16× upsampling ratios. Table 1 also reports the

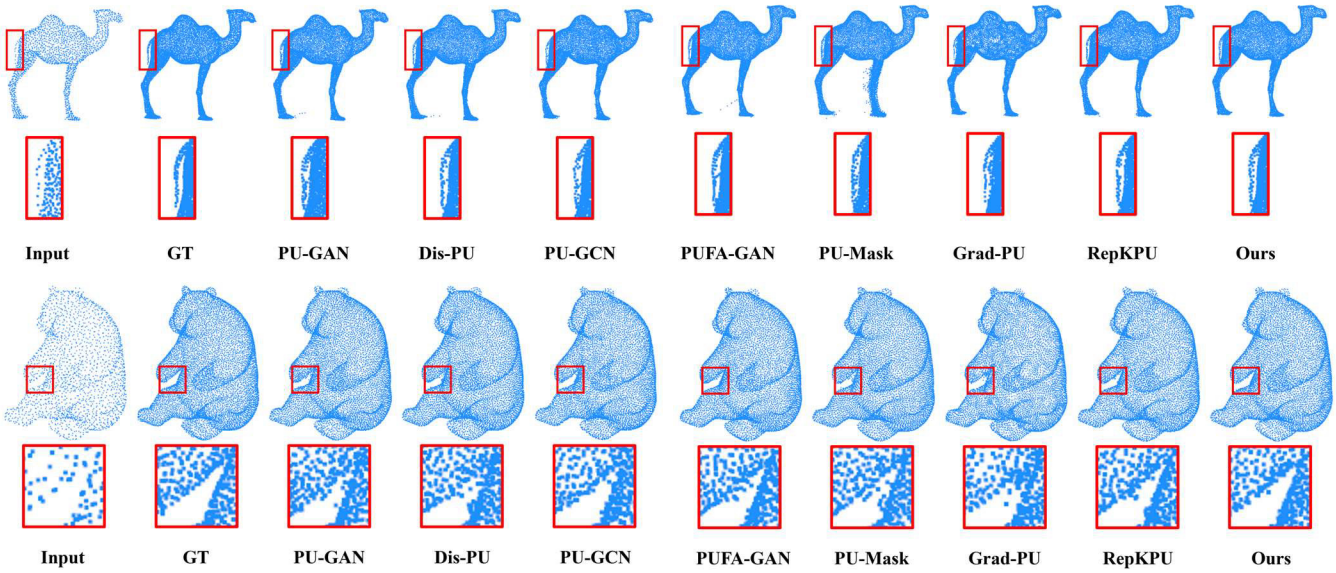

FIGURE 3. Qualitative comparison of 4x upsampled point clouds on the PU-GAN dataset. Red boxes indicate zoomed-in regions.

TABLE 3. Quantitative results of arbitrary-scale upsampling on the PU-GAN dataset.

Rates	Grad-PU[26]			RepKPU[27]			Ours		
	CD↓ $\times 10^3$	HD↓ $\times 10^2$	P2F↓ $\times 10^3$	CD↓ $\times 10^3$	HD↓ $\times 10^2$	P2F↓ $\times 10^3$	CD↓ $\times 10^3$	HD↓ $\times 10^2$	P2F↓ $\times 10^3$
5 ×	0.239	2.582	2.445	**0.225**	**2.223**	**1.879**	**0.223**	**2.325**	**1.791**
6 ×	0.232	2.548	2.900	**0.205**	**2.092**	**1.872**	**0.205**	**2.270**	**1.782**
7 ×	0.225	2.669	3.336	**0.183**	**1.985**	**1.775**	**0.179**	**2.204**	**1.693**
11 ×	0.223	3.504	4.946	**0.130**	**1.824**	**1.801**	**0.129**	**1.947**	**1.751**
13 ×	0.227	4.139	5.690	**0.118**	**1.810**	**1.907**	**0.111**	**1.824**	**1.873**
15 ×	0.253	4.541	6.396	**0.103**	**1.849**	**2.028**	**0.101**	**1.842**	**2.013**

We compare our method with Grad-PU and RepKPU, and highlight the best and second-best values in bold red and blue, respectively.

inference speed (ms/pcd) and the number of network parameters. As shown in Table 1, our method achieves the best performance in terms of CD and P2F at 4× upsampling, while ranking third in HD and second in inference speed. At 16× upsampling, it ranks first in P2F, second in CD, and third in HD. These results indicate that our method is particularly effective at reconstructing accurate and smooth surfaces from sparse inputs, as reflected by its strong performance in both average (CD) and surface-level (P2F) accuracy metrics. We further conducted experiments on the PU1K dataset with a 4× upsampling ratio. Unlike PU-GAN, which mainly contains simple objects, PU1K includes a wide variety of complex 3D models. As reported in Table 2, our method outperforms all prior methods across all metrics. This demonstrates its strong ability to capture fine details, edges, and curved surfaces, and generalize well to diverse and unseen geometries.

E. QUALITATIVE EVALUATION

In addition to the quantitative evaluations presented in Section IV-D, we provide qualitative visual results to further demonstrate the effectiveness of our method. Fig. 3 shows the upsampled point clouds generated by our model trained on the PU-GAN dataset. The results indicate that our method effectively preserves fine-grained geometric details.

To further assess surface quality, Fig. 4 presents both the upsampled point clouds and their corresponding mesh reconstructions using the Ball Pivoting Algorithm

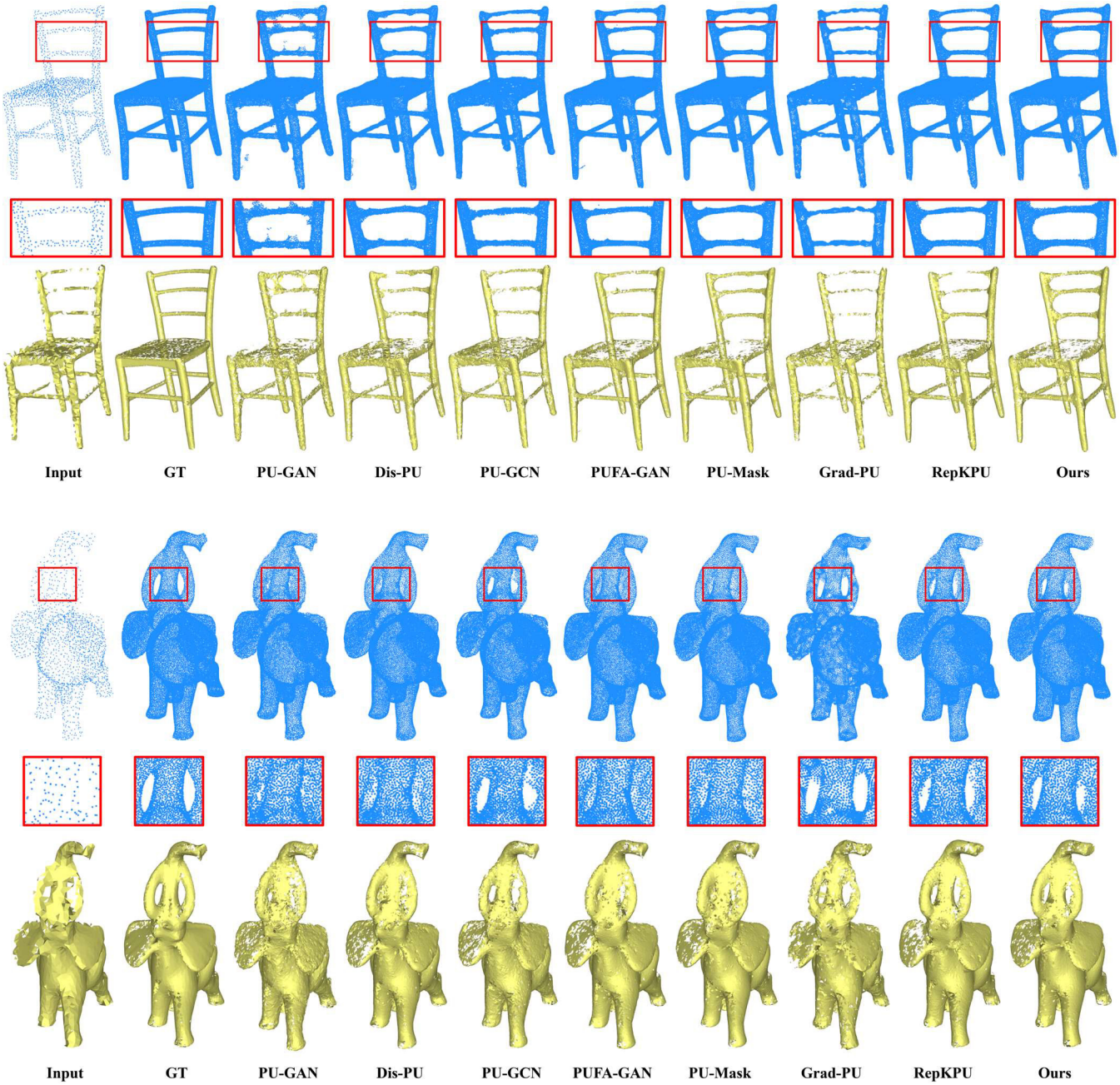

FIGURE 4. Surface mesh reconstruction results from 16× upsampled point clouds on the PU-GAN dataset. Red boxes indicate zoomed-in point cloud regions.

(BPA) [47], based on 16× upsampling of the PU-GAN dataset. The comparisons highlight that our method yields smoother and more geometrically accurate surfaces, particularly around curved regions and object boundaries. As noted in Section IV-C, the Hausdorff Distance (HD) is not an average-based measure and thus does not fully capture the overall distribution of the upsampled point cloud. This explains why PU-GCN, despite showing competitive HD values, does not produce visually faithful reconstructions compared to our method. The red boxes indicate zoomed-in regions of the point clouds.

F. ARBITRARY UPSAMPLING TEST

We also conduct experiments under arbitrary upsampling ratios to evaluate the flexibility of our method. Specifically, we perform upsampling at six different ratios: 5×, 6×, 7×, 11×, 13×, and 15×. For comparison, we include Grad-PU and RepKPU, which not only achieved the best overall performance among existing methods in CD and P2F metrics (see

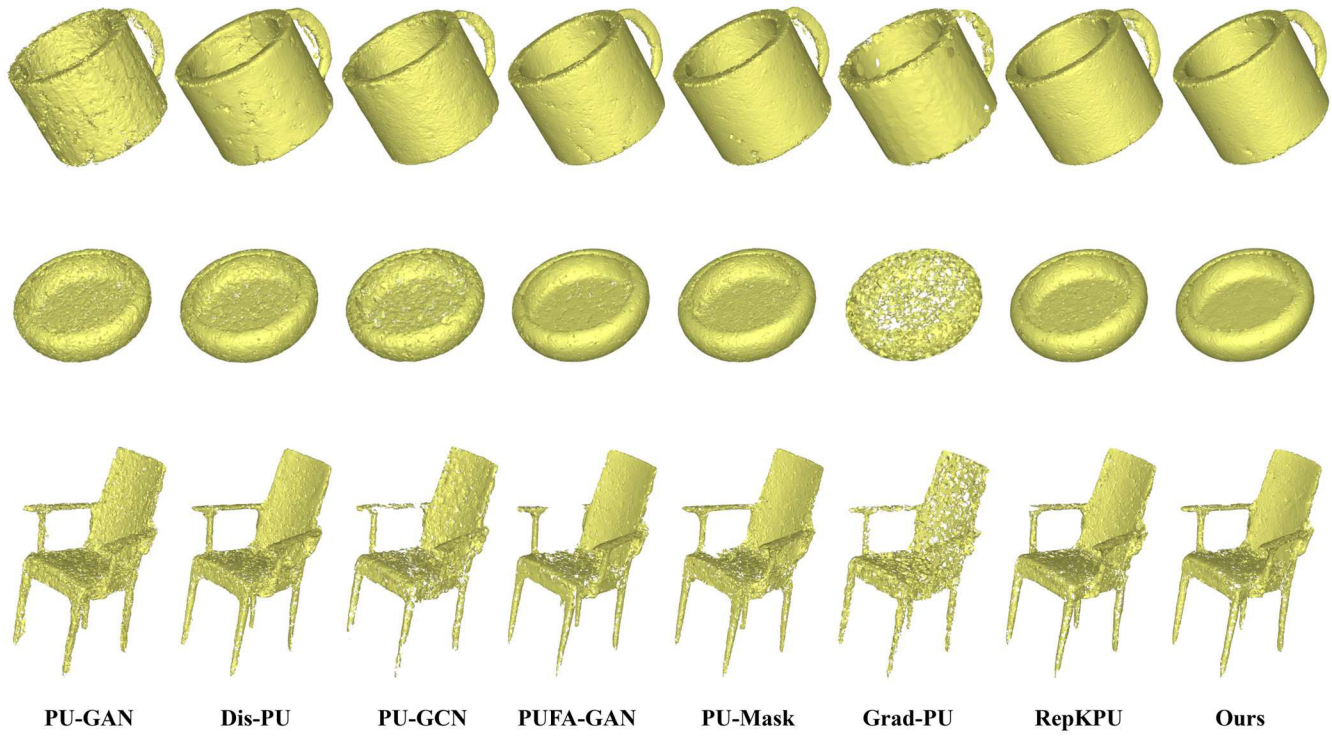

FIGURE 5. Mesh reconstruction results on unseen shapes from the ShapeNetCore dataset using 16× upsampled point clouds.

TABLE 4. Quantitative results under random noise on the PU-GAN dataset.

Methods	CD↓ $\times 10^3$	HD↓ $\times 10^2$	P2F↓ $\times 10^3$
PUFA-GAN[21]	0.288	3.634	**3.562**
PU-Mask[28]	0.291	4.279	3.662
Grad-PU[26]	0.320	**2.947**	4.162
RepKPU[27]	**0.286**	**3.165**	3.611
Ours	**0.282**	3.679	**3.530**

To assess robustness, our method is compared with recent upsampling methods. We highlight the best and second-best values in bold red and blue, respectively.

TABLE 5. Ablation study on the PU-GAN dataset.

Methods	CD↓ $\times 10^3$	HD↓ $\times 10^2$	P2F↓ $\times 10^3$
Ours - no HCD loss	0.324	0.272	0.875
Ours - no Adaptive KP-QG Module	0.329	0.270	0.933
Ours	0.316	0.265	0.864

Table 1), but also explicitly address arbitrary-scale upsampling in their design. The experiments are conducted on the PU-GAN dataset. Following the same setup as in the 4× and 16× upsampling tasks, we generate 2,048 input points from the original test mesh using Poisson disk sampling and create ground truth point clouds with corresponding target sizes. The output point cloud sizes for each ratio are 10,240, 12,288, 14,336, 22,528, 26,624, and 30,720 points, respectively. We adopt the Fixed-to-Arbitrary (F2A) framework introduced in RepKPU for arbitrary upsampling. As shown in Table 3, our method achieves consistently lower P2F values across most ratios, demonstrating strong generalization and high-fidelity surface reconstruction even under non-standard upsampling factors. We further evaluate the robustness of our method by injecting random noise into the input point clouds.

G. ROBUST TEST

We further evaluate the robustness of our method by injecting random noise into the input point clouds. The noise is added only to the inputs, while the ground truth point clouds remain fixed with 8,192 points. Each input contains 2,048 points. The random noise is sampled from a uniform distribution with a noise level of 0.01. As shown in Table 4, our method

demonstrates superior performance in terms of P2F, indicating strong robustness against input perturbations.

H. QUALITATIVE EVALUATION ON UNSEEN DATASET

To assess the generalization capability of our method, we evaluate it on an external dataset not used during training, namely ShapeNetCore. As shown in Fig. 5, we compare the upsampled point clouds generated by different methods from the same input point clouds, along with their corresponding mesh reconstructions obtained via the Ball Pivoting Algorithm (BPA). Mesh reconstructions from the low-resolution input point clouds are not included, since the sparse inputs fail to produce meaningful surfaces and lead to severely distorted meshes. The input point clouds are derived from ShapeNetCore meshes using Poisson disk sampling. The model used for inference was trained on the PU-GAN dataset. The results clearly show that our method produces high-fidelity point clouds and reconstructs accurate surface geometries even on previously unseen shapes.

I. ABLATION STUDY

To validate the effectiveness of the two core components (Adaptive KP-Queries Generation Module, Hyper Chamfer Distance loss) of our proposed method, we conduct an ablation study. In the setting Ours – no HD loss, we apply the Adaptive KP-Queries Generation Module while excluding the Hyper Chamfer Distance loss. In contrast, Ours – no Adaptive KP-QG Module uses the Hyper Chamfer Distance loss without the Adaptive KP-Queries Generation Module. The performance results for both settings are presented in Table 5.

As shown in Ours – no HD loss, incorporating the adaptive projection function $h_{adaptive}$ enables the model to better capture local geometric structures, leading to improved performance. In contrast, the results of Ours – no Adaptive KP-QG Module indicate that using Hyper Chamfer Distance loss alone does not guarantee significant gains. However, when combined with the Adaptive KP-Queries Generation Module, the Hyper Chamfer Distance loss proves highly effective, particularly in modeling complex surface geometries.

V. CONCLUSION

This study aims to improve surface reconstruction performance in point cloud upsampling by proposing an attention-based adaptive projection module and introducing the Hyper Chamfer Distance loss into the upsampling task. Experimental results demonstrate that the proposed approach achieves more accurate point reconstruction, especially in curved and complex regions, compared to previous methods. Furthermore, the method maintains consistent performance across various upsampling scales and under noisy conditions, highlighting its robustness and practical applicability.

However, the manual adjustment of the attention-based weighting factor λ depending on the dataset remains a limitation. This could be addressed in future work by adopting learnable or data-driven weight adjustment strategies. The proposed module also shows strong potential for integration into other upsampling frameworks or related tasks such as mesh reconstruction, opening up promising directions for future research.

REFERENCES

[1] Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, and A. Markham, "RandLA-Net: Efficient semantic segmentation of large-scale point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2020, pp. 11105–11114, doi: 10.1109/CVPR42600.2020.01112.

[2] B. Ma, Y.-S. Liu, M. Zwicker, and Z. Han, "Surface reconstruction from point clouds by learning predictive context priors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, New Orleans, LA, USA, Jun. 2022, pp. 6316–6327, doi: 10.1109/CVPR52688.2022.00622.

[3] Y. Cui, R. Chen, W. Chu, L. Chen, D. Tian, Y. Li, and D. Cao, "Deep learning for image and point cloud fusion in autonomous driving: A review," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 722–739, Feb. 2022, doi: 10.1109/TITS.2020.3023541.

[4] F. Pomerleau, F. Colas, and R. Siegwart, "A review of point cloud registration algorithms for mobile robotics," *Found. Trends Robot.*, vol. 4, no. 1, pp. 1–104, 2015, doi: 10.1561/2300000035.

[5] M. Alexa, J. Behr, D. Cohen-Or, S. Fleishman, D. Levin, and C. T. Silva, "Computing and rendering point set surfaces," *IEEE Trans. Vis. Comput. Graphics*, vol. 9, no. 1, pp. 3–15, Jan. 2003.

[6] Y. Lipman, D. Cohen-Or, D. Levin, and H. Tal-Ezer, "Parameterization-free projection for geometry reconstruction," in *Proc. ACM SIGGRAPH Papers*, Jul. 2007, p. 22.

[7] H. Huang, D. Li, H. Zhang, U. Ascher, and D. Cohen-Or, "Consolidation of unorganized point clouds for surface reconstruction," *ACM Trans. Graph.*, vol. 28, no. 5, pp. 1–7, Dec. 2009.

[8] H. Huang, S. Wu, M. Gong, D. Cohen-Or, U. Ascher, and H. Zhang, "Edge-aware point set resampling," *ACM Trans. Graph.*, vol. 32, no. 1, pp. 1–12, Jan. 2013.

[9] R. Preiner, O. Mattausch, M. Arikan, R. Pajarola, and M. Wimmer, "Continuous projection for fast L_1 reconstruction," *ACM Trans. Graph.*, vol. 33, no. 4, pp. 1–13, Jul. 2014.

[10] S. Wu, H. Huang, M. Gong, M. Zwicker, and D. Cohen-Or, "Deep points consolidation," *ACM Trans. Graph.*, vol. 34, no. 6, pp. 1–13, Nov. 2015.

[11] L. Yu, X. Li, C.-W. Fu, D. Cohen-Or, and P.-A. Heng, "PU-net: Point cloud upsampling network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2790–2799.

[12] W. Yifan, S. Wu, H. Huang, D. Cohen-Or, and O. Sorkine-Hornung, "Patch-based progressive 3D point set upsampling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 5951–5960.

[13] R. Li, X. Li, C.-W. Fu, D. Cohen-Or, and P.-A. Heng, "PU-GAN: A point cloud upsampling adversarial network," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 7202–7211.

[14] Y. Qian, J. Hou, S. Kwong, and Y. He, "PUGeo-Net: A geometry-centric network for 3D point cloud upsampling," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Nov. 2020, pp. 752–769.

[15] R. Li, X. Li, P.-A. Heng, and C.-W. Fu, "Point cloud upsampling via disentangled refinement," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 344–353.

[16] G. Qian, A. Abualshour, G. Li, A. Thabet, and B. Ghanem, "PU-GCN: Point cloud upsampling using graph convolutional networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 11678–11687.

[17] S. Qiu, S. Anwar, and N. Barnes, "PU-transformer: Point cloud upsampling transformer," in *Proc. Asian Conf. Comput. Vis. (ACCV)*, Dec. 2023, pp. 326–343.

[18] Y. Qian, J. Hou, S. Kwong, and Y. He, "Deep magnification-flexible upsampling over 3D point clouds," *IEEE Trans. Image Process.*, vol. 30, pp. 8354–8367, 2021.

[19] H. Liu, H. Yuan, R. Hamzaoui, W. Gao, and S. Li, "PU-refiner: A geometry refiner with adversarial learning for point cloud upsampling," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 2270–2274.

[20] H. Zhou, Y. Cao, W. Chu, J. Zhu, T. Lu, Y. Tai, and C. Wang, "SeedFormer: Patch seeds based point cloud completion with upsample transformer," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 416–432.

[21] H. Liu, H. Yuan, J. Hou, R. Hamzaoui, and W. Gao, "PUFA-GAN: A frequency-aware generative adversarial network for 3D point cloud upsampling," *IEEE Trans. Image Process.*, vol. 31, pp. 7389–7402, 2022.

[22] W. Feng, L. Jin, H. Cai, X. Luo, and J. Zhang, "Neural points: Point cloud representation with neural fields for arbitrary upsampling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 18612–18621.

[23] A. Mao, Z. Du, J. Hou, Y. Duan, Y.-J. Liu, and Y. He, "PU-flow: A point cloud upsampling network with normalizing flows," *IEEE Trans. Vis. Comput. Graphics*, vol. 29, no. 12, pp. 4964–4977, Dec. 2023, doi: 10.1109/TVCG.2022.3196334.

[24] A. Akhtar, Z. Li, G. V. d. Auwera, L. Li, and J. Chen, "PU-dense: Sparse tensor-based point cloud geometry upsampling," *IEEE Trans. Image Process.*, vol. 31, pp. 4133–4148, 2022.

[25] C. Long, W. Zhang, R. Li, H. Wang, Z. Dong, and B. Yang, "PC^2-PU: Patch correlation and point correlation for effective point cloud upsampling," in *Proc. 30th ACM Int. Conf. Multimedia*, Oct. 2022, pp. 2191–2201.

[26] Y. He, D. Tang, Y. Zhang, X. Xue, and Y. Fu, "Grad-PU: Arbitrary-scale point cloud upsampling via gradient descent with learned distance functions," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 5354–5363.

[27] Y. Rong, H. Zhou, K. Xia, C. Mei, J. Wang, and T. Lu, "RepKPU: Point cloud upsampling with kernel point representation and deformation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 21050–21060.

[28] H. Liu, H. Yuan, R. Hamzaoui, Q. Liu, and S. Li, "PU-mask: 3D point cloud upsampling via an implicit virtual mask," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 7, pp. 6489–6502, Jul. 2024.

[29] C. R. Qi, Y. Li, H. Su, and L. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5099–5108.

[30] A. Kazi, S. Shekarforoush, S. A. krishna, H. Burwinkel, G. Vivar, K. Kortüm, S. Ahmadi, S. Albarqouni, and N. Navab, "InceptionGCN: Receptive field aware graph convolutional network for disease prediction," in *Proc. Int. Conf. Inf. Process. Med. Imag.*, 2019, pp. 73–85.

[31] N. Engel, V. Belagiannis, and K. Dietmayer, "Point transformer," *IEEE Access*, vol. 9, pp. 134826–134840, 2021.

[32] H. Thomas, C. R. Qi, J. Deschaud, B. Marcotegui, F. Goulette, and L. Guibas, "KPConv: Flexible and deformable convolution for point clouds," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 6410–6419.

[33] H. Fan, Y. Yang, and M. Kankanhalli, "Point 4D transformer networks for spatio-temporal modeling in point cloud videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 14204–14213.

[34] Y. Eldar, M. Lindenbaum, M. Porat, and Y. Y. Zeevi, "The farthest point strategy for progressive image sampling," *IEEE Trans. Image Process.*, vol. 6, no. 9, pp. 1305–1315, Sep. 1997.

[35] K. Low, "Linear least-squares optimization for point-to-plane ICP surface registration," Univ. North Carolina Chapel Hill, Chapel Hill, NC, USA, Tech. Rep. TR04-004, 2004, pp. C1–C3, vol. 4, no. 10.

[36] F. Lin, Y. Yue, S. Hou, X. Yu, Y. Xu, K. D. Yamada, and Z. Zhang, "Hyperbolic chamfer distance for point cloud completion," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Paris, France, Oct. 2023, pp. 14549–14560, doi: 10.1109/iccv51070.2023.01342.

[37] H. Fan, H. Su, and L. Guibas, "A point set generation network for 3D object reconstruction from a single image," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2463–2471.

[38] A. Montanaro, D. Valsesia, and E. Magli, "Rethinking the compositionality of point clouds through regularization in the hyperbolic space," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 33741–33753.

[39] O.-E. Ganea, G. Bécigneul, and T. Hofmann, "Hyperbolic neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2018, pp. 5350–5360.

[40] R. Shimizu, Y. Mukuta, and T. Harada, "Hyperbolic neural networks++," 2020, *arXiv:2006.08210*.

[41] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic graph CNN for learning on point clouds," *ACM Trans. Graph.*, vol. 38, no. 5, pp. 1–12, Oct. 2019.

[42] R. Labaca-Castro, "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023, pp. 73–76.

[43] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[44] F. Bernardini, J. Mittleman, H. Rushmeier, C. Silva, and G. Taubin, "The ball-pivoting algorithm for surface reconstruction," *IEEE Trans. Vis. Comput. Graphics*, vol. 5, no. 4, pp. 349–359, Oct. 1999.

[45] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu, "ShapeNet: An information-rich 3D model repository," 2015, *arXiv:1512.03012*.

[46] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent.*, 2014, pp. 1–15.

[47] C. Yuksel, "Sample elimination for generating Poisson disk sample sets," *Comput. Graph. Forum*, vol. 34, no. 2, pp. 25–32, May 2015.

[48] M. Berger, J. A. Levine, L. G. Nonato, G. Taubin, and C. T. Silva, "A benchmark for surface reconstruction," *ACM Trans. Graph.*, vol. 32, no. 2, pp. 1–17, Apr. 2013.

JONG-SU WON was born in Pyeongtaek, South Korea, in 2000. He received the B.S. degree in electrical engineering from Sejong University, Seoul, South Korea, where he is currently pursuing the M.S. degree. His research interests include computer vision, virtual reality (VR), and machine learning.

JONG-KI HAN was born in Seoul, South Korea, in 1968. He received the B.S., M.S., and Ph.D. degrees in electrical engineering from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 1992, 1994, and 1999, respectively. From 1999 to 2001, he was a member of Technical Staff with the Corporate Research and Development Center, Samsung Electronics Company, Suwon, South Korea. He is currently a Professor with the Department of Electrical Engineering, Sejong University, Seoul. His research interests include image and video coding using high efficiency video coding (HEVC), versatile video coding (VVC), and image coding and processing for 360° virtual reality (VR). He has been participating in the standardization of HEVC and VVC, since 2009 and 2016, respectively.